

# Time-Series Analysis

Economists and electrical engineers have long been trying to predict the next signal in a time series, which is exactly what traders try to do as well. This chapter is an introduction to the tools well known in econometrics and signal processing, and which have found wide acceptance in the quantitative investment community.

59

You may already have seen some time-series analysis techniques in action in my previous books (Chan, 2009 and 2013), as a way to test for stationarity or cointegration of price series. But these are just parts of a general package of linear modeling techniques with acronyms like ARIMA, VAR, or VEC. Likewise, almost every technical trader has tried moving averages as a way to filter out the noise in price series. But have they tried many of the advanced signal processing filters such as the Kalman filter?

Time-series techniques are most useful in markets where fundamental information and intuition are either lacking or not particularly useful for short-term predictions. Currencies and bitcoins fit this bill. Professor Lyons (2001) wrote that “... the proportion of monthly exchange rate changes our textbook models can explain is essentially zero.” We will mention a few examples of using time-series techniques to predict currency returns in this chapter, and leave the bitcoin examples to Chapter 7. But just as technical analysis can be useful for stock trading despite the abundance of fundamental information there, we will describe examples where time-series analysis can be applied to stocks.

Unlike other books on time-series analysis, we will not be discussing the inner workings of these techniques, but focus solely on how we can use

ready-made software packages to make predictions. Most of the examples are implemented using the MATLAB Econometrics Toolbox, but *R* users can find similar functions in the *forecast*, *vars*, and *dlm* packages.

## ■ AR(*p*)

The simplest model in time-series analysis is AR(1). It is just a linear regression model that relates the price in one bar to the next:

$$Y(t) - \mu = \phi(Y(t-1) - \mu) + \varepsilon(t) \quad (3.1)$$

where  $Y(t)$  is the price at time  $t$ ,  $\phi$  is the (auto)regression coefficient, and  $\varepsilon$  is Gaussian noise with zero mean, sometimes called innovation. Hence, the name *auto-regressive* process. A time series is called weakly<sup>1</sup> stationary if its mean and variance are constant in time, and AR(1) is weakly stationary if  $|\phi| < 1$  (the proof is left as an exercise). A weakly stationary time series is also mean reverting (Chan, 2013). If  $|\phi| > 1$ , the time series will trend. If  $\phi = 1$ , we have a random walk. To estimate  $\phi$ , we use the *arima* and *estimate* functions in the Econometrics Toolbox.

```
model_ar1=arima(1, 0, 0) % assumes an AR(1) with unknown parameters
model_ar1_estimates=estimate(model_ar1, cl);
```

The function *arima*( $p, d, q$ ) reduces to an AR(1) model if we set  $p = 1$  and  $d = 0$  (We will discuss the more general version in the next section.) The *estimate* function just applies maximum likelihood estimation to find the parameters for the AR(1) model based on the input price series. Applying it to the one-minute midprice bars of AUD.USD from July 24, 2007 to August 3 2015 returns an estimate of  $\phi = 0.99997$ , with a standard error of 0.00001.<sup>2</sup> We conclude that though AUD.USD is very weakly stationary, it is very close to a random walk. Note that we tested on midprices instead of trade prices to reduce bid–ask bounce, which tends to produce phantom mean-reversion that cannot really be traded on.

Generalizing slightly from AR(1), we can consider AR( $p$ ), represented by

$$Y(t) = \mu + \phi_1 Y(t-1) + \phi_2 Y(t-2) + \cdots + \phi_p Y(t-p) + \varepsilon(t). \quad (3.2)$$

You can see that this is just a multiple regression model with the price at time  $t$  as the dependent (response) variable and past prices up to a lag of  $p$  bars as independent (predictor) variables. But introducing  $p$  as an additional parameter means that we can find the optimal  $p$  that gives the best fit of the AR( $p$ ) model to our data. As in many statistical models, we will use the

Bayesian information criterion (BIC) that is proportional to the negative log likelihood of the model but with an additional term that is proportional to  $p$ , which penalizes complexity. Our objective is to minimize BIC, and we do this by a brute-force exhaustive search:<sup>3</sup>

```
LOGL=zeros(60, 1); % log likelihood for up to 60 lags (1 hour)
P=zeros(size(LOGL)); % p values

for p=1:length(P)
    model=arima(p, 0, 0);
    [~,~,logL] = estimate(model, mid(trainset), 'print', false);
    LOGL(p) = logL;
    P(p) = p;
end

% Has P+1 parameters, including constant
[~, bic]=aicbic(LOGL, P+1, length(mid(trainset)));

[~, pMin]=min(bic)

model=arima(pMin, 0, 0) % assumes an AR(pMin) with unknown
    parameters
```

In the above code fragment, *mid* is the array that contains the midprices.

Once we have decided on the best estimate of  $p$ , we can apply the *estimate* function to it to find the coefficients  $\mu, \phi_1, \phi_2, \dots, \phi_p$ :

```
fit=estimate(model, mid);
```

Applying these functions to AUD.USD on one-minute midprice bars from July 24, 2007, to August 12, 2014, yields  $p = 10$  as the optimal value, with the coefficients noted in Table 3.1.

We can now use this AR(10) model for prediction on the out-of-sample data set from August 12, 2014, to August 3, 2015.

```
yF=NaN(size(mid));
for t=testset(1):size(mid, 1)
    [y, ~]=forecast(fit, 1, 'Y0', mid(t-pMin+1:t)); % Need only
        most recent pMin data points for prediction
    yF(t)=y(end);
end
```

Note that  $yF(t)$  is the forecast made with data up to time  $t$ ; hence, it is actually the predicted price for time  $t + 1$ . Once the next bar prediction has

**TABLE 3.1** Coefficients of an AR(10) Model Applied to AUD.USD

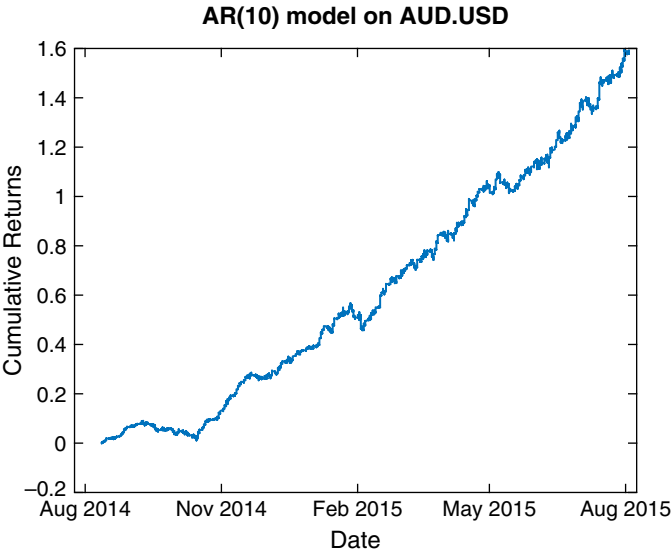
| Coefficient | Value        | Standard Error |
|-------------|--------------|----------------|
| $\mu$       | 1.37196e-06  | 8.65314e-07    |
| $\phi_1$    | 0.993434     | 0.000187164    |
| $\phi_2$    | -0.00121205  | 0.000293356    |
| $\phi_3$    | -0.000352717 | 0.000305831    |
| $\phi_4$    | 0.000753222  | 0.000354121    |
| $\phi_5$    | 0.00662641   | 0.000358673    |
| $\phi_6$    | -0.00224118  | 0.000330092    |
| $\phi_7$    | -0.00305157  | 0.000365348    |
| $\phi_8$    | 0.00351317   | 0.000394538    |
| $\phi_9$    | -0.00154844  | 0.000398956    |
| $\phi_{10}$ | 0.00407798   | 0.000281821    |

been made, we can use it to generate trading signals: Simply buy when the predicted price is higher than the current price, and sell when it is lower:

```
deltaYF=yF-mid;

pos=zeros(size(mid));
pos(deltaYF > 0)=1;
pos(deltaYF < 0)=-1;
```

This strategy yields an annualized return of 158 percent on the out-of-sample set. See Figure 3.1 for its equity curve. To realize such amazing



**FIGURE 3.1** AR(10) trading strategy applied to AUD.USD

returns, one has to be able to execute at midprice; hence, a low latency execution program that manages limit orders is necessary.

## ■ ARMA( $p, q$ )

From our application of AR( $p$ ) to AUD.USD, we see that the best fit requires 10 lags. This high number of lags is quite common for AR( $p$ ) models: They are trying to compensate for the simplicity of the model structure with a larger number of terms. A small extension of the AR model to include  $q$  lagged noise terms will often reduce the number of lags necessary. This is called the ARMA( $p, q$ ) model, or an auto-regressive moving average process, where the  $q$  lagged noise terms are described as a *moving average*:

$$Y(t) = \mu + \phi_1 Y(t-1) + \phi_2 Y(t-2) + \cdots + \phi_p Y(t-p) + \varepsilon(t) + \theta_1 \varepsilon(t-1) + \cdots + \theta_q \varepsilon(t-q) \quad (3.3)$$

Finding the best values of the  $p$  and  $q$  and the coefficient of each term in equation 3.3 is similar to the procedure we took for AR( $p$ ), but because we are now doing exhaustive search over two variables, we need nested for-loops:

```
LOGL=-Inf(10, 9); % log likelihood for up to 10 p and 9 q
               (10 minutes)
PQ=zeros(size(LOGL)); % p and q values

for p=1:size(PQ, 1)
    for q=1:size(PQ, 2)
        model=arima(p, 0, q);
        [~,~,logL] = estimate(model, mid(trainset),'print',false);
        LOGL(p, q) = logL;
        PQ(p, q) = p+q;
    end
end
```

For each  $p$  and  $q$ , we save the log likelihood in  $LOGL(p, q)$ , and  $p + q$  in  $PQ(p, q)$ , the latter because it is used as a penalty term when minimizing BIC. How do we identify the optimal  $p$  and  $q$  that minimizes BIC from the  $LOGL$  and  $PQ$  matrices? We have to turn them into one-dimensional vectors, apply the *aicbic* function, and then use the *min* function:

```
% Has p+q+1 parameters, including constant
LOGL_vector = reshape(LOGL, size(LOGL, 1)*size(LOGL, 2), 1);
PQ_vector = reshape(PQ, size(LOGL, 1)*size(LOGL, 2), 1);
```

```
[~, bic]=aicbic(LOGL_vector, PQ_vector+1, length(mid(trainset)));
[bicMin, pMin]=min(bic)
```

Finally, we have to turn the one-dimensional BIC vector back into a two-dimensional array, but with only the cell corresponding to the minimum value populated, in order to facilitate easy visual identification of the row (corresponding to  $p$ ) and column (corresponding to  $q$ ) numbers of that cell:

```
bic(:)=NaN;
bic(pMin)=bicMin;
bic=reshape(bic,size(LOGL))
```

All these procedures are contained in the program *buildARMA\_findPQ\_AUDUSD.m*. The output for AUD.USD looks like the following:

```
bic =

    1.0e+07 *

Columns 1 through 4

      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN

Columns 5 through 8

      NaN      NaN      NaN      NaN
-3.469505397473728      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
      NaN      NaN      NaN      NaN
```

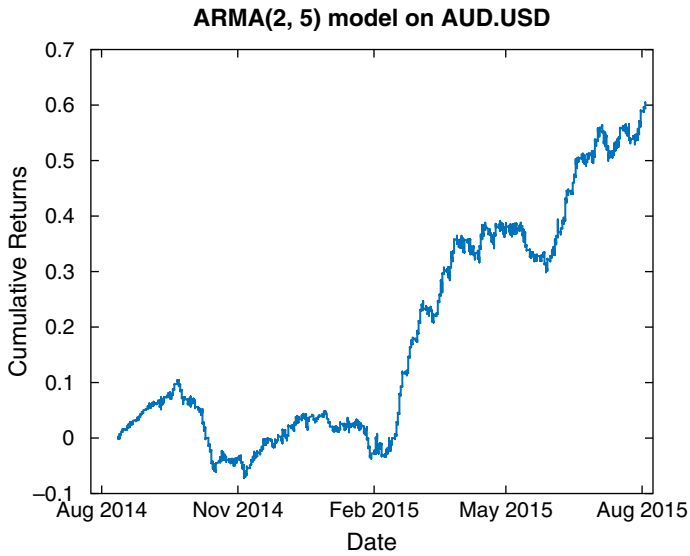
NaN  
NaN  
NaN  
NaN  
NaN  
NaN  
NaN  
NaN  
NaN

where we easily determine that the cell with the minimum BIC corresponds to  $p = 2$  and  $q = 5$ . These are indeed shorter lags than the  $p = 10$  we used in the  $AR(p)$  model. Plugging in these values to the *arima* function and then applying the *estimate* function on the  $ARMA(2, 5)$  model as we did in the section on  $AR(p)$  yields the coefficients shown in Table 3.2.

One should note that  $|\phi_1|$  is now definitely smaller than 1, indicating strong mean reversion. However, using the *forecast* functions to generate trading signals as before actually decreases the out-of-sample annualized return from 158 percent to 60 percent. The added complexity of using moving average has not paid off in this case. The equity curve is shown in Figure 3.2. The backtest program is available as `buildARMA_AUDUSD.m`.

You may wonder why the function we used for the  $AR(p)$  and  $ARMA(p, q)$  models are called *arima*. You may also wonder why we focus on predicting prices rather than returns. The answer to both questions can be understood by studying the  $ARIMA(p, d, q)$  model.

| TABLE 3.2 Coefficients of an ARMA(2, 5) Model Applied to AUD.USD |             |                |
|------------------------------------------------------------------|-------------|----------------|
| Coefficient                                                      | Value       | Standard Error |
| $\mu$                                                            | 2.80383e-06 | 4.58975e-06    |
| $\phi_1$                                                         | 0.649011    | 0.000249771    |
| $\phi_2$                                                         | 0.350986    | 0.000249775    |
| $\theta_1$                                                       | 0.345806    | 0.000499929    |
| $\theta_2$                                                       | −0.00906282 | 0.000874713    |
| $\theta_3$                                                       | −0.0106082  | 0.000896239    |
| $\theta_4$                                                       | −0.0102606  | 0.0010664      |
| $\theta_5$                                                       | −0.00251154 | 0.000910359    |



**FIGURE 3.2** ARMA(2, 5) trading strategy applied to AUD.USD

ARIMA( $p, d, q$ ) stands for autoregressive *integrated* moving average. Let's just concern ourselves with  $d = 1$ , the simplest and the most common case in finance. If  $Y(t)$  is an ARIMA( $p, 1, q$ ) model, it implies that  $\Delta Y(t)$  is an ARMA( $p, q$ ), where  $\Delta Y(t) = Y(t) - Y(t - 1)$ . We can understand this even better if  $Y(t)$  represents log price instead of price. If this is the case, then using ARMA( $p, q$ ) to model the log returns is equivalent to using ARIMA( $p, 1, q$ ) to model the log prices.

Would it be advantageous to model log returns  $\Delta Y$  instead of  $Y$  using ARMA( $p, q$ )? It would be, if we can further reduce the lags  $p$  and  $q$  from the ones obtained when modeling prices (or log prices) using ARMA( $p, q$ ). Unfortunately, I have never found that to be true. For example, modeling the log of AUD.USD time series using ARIMA( $p, 1, q$ ) gives  $p = 1$ , and  $q = 9$ .

The equivalence of an ARIMA( $p, 1, q$ ) model on log prices to an ARIMA( $p, 0, q$ ) model on log returns should not be confused with the statement that an ARMA( $p, q$ ) = ARIMA( $p, 0, q$ ) model on log prices is equivalent to some ARMA( $p', q'$ ) model on log returns. The latter statement is false. An ARMA model in  $\Delta Y$ 's can always be transformed into an ARMA model in  $Y$ 's. But an ARMA model for  $Y$  cannot always be transformed into an ARMA model for  $\Delta Y$ . This is because an ARMA model for  $\Delta Y$  can only have  $\Delta Y$  as independent variables, whereas an ARMA model for  $Y$  can have both  $\Delta Y$  (which is just the difference of two  $Y$ s) and



$Y$  as independent variables. Hence, a model for  $Y$  is more flexible and gives better results. If we want to have a model for  $\Delta Y$  that has both  $\Delta Y$ s and  $Y$ s as independent variables, we have to use a VEC( $p$ ) model, to be discussed at the end of the next section on VAR( $p$ ).

## ■ VAR( $p$ )

The simple autoregressive model AR( $p$ ) in equation 3.2 can be easily generalized to  $m$  multivariate time series. This generalized model is called a *vector autoregressive* model, or VAR( $p$ ). All we need to do is to interpret the autoregressive coefficients  $\phi$  as  $m \times m$  matrices, and allow the noises  $\epsilon(t)$ , which are  $m$ -vectors to have nonzero cross-sectional correlations but zero serial correlations. This means that  $\epsilon_i(t)$  is not correlated with  $\epsilon_j(s)$ , for any  $t \neq s$ , but  $\epsilon_i(t)$  could be correlated with  $\epsilon_j(t)$ . Since the autoregressive coefficient matrices relate the current price of every time series to the lagged prices of all time series, VAR model is particularly suitable for modeling financial instruments that have correlated returns, such as a portfolio of stocks within the same industry group. We will focus on the computer hardware group within the S&P 500 Index on January 3, 2007, which consists of the tickers AAPL, EMC, HPQ, NTAP, and SNDK. To eliminate spurious mean-reversion effects due to bid-ask bounce, we will use midprices at market close provided by the Center for Research of Security Prices (CRSP) from January 3, 2007, to December 31, 2013.

As in the section on AR( $p$ ), we first need to determine the optimal lag  $p$ . We will use the first six years of data as training set for this determination. There are only minor differences in the codes required:<sup>4</sup>

```
for p=1:length(P)
    model=vgxset('n', size(mid, 2), 'nAR', p, 'Constant', true);
        % with additive offset
    [model,EstStdErrors,logL,W] = vgxvarx(model,mid(trainset, :));
    [NumParam,~] = vgxcount(model);
    LOGL(p) = logL;
    P(p) = NumParam;
end
```

It is gratifying that we find  $p = 1$  minimizes BIC (simpler models are usually better), and this is a typical result for most industry groups. Once this is decided, the other parameters of the model can be determined by the function *vgxvarx*, which is the equivalent of the *estimate* function for ARIMA

models. Using the same training set, the constant offsets, autoregressive coefficients, and the covariance of the noise terms are noted in Table 3.3. (In this table, in contrast to Table 3.1 or 3.2, the subscripts refer to the stocks instead of number of time lags.)

To make predictions using this model on the out-of-sample data in 2013, use the *vgxpred* function, which is similar to the *forecast* function for ARIMA.

```
pMin=1;
yF=NaN(size(mid));
for t=testset(1):size(mid, 1)
    FY = vgxpred(model,1, [], mid(t-pMin+1:t, :));
    yF(t, :)=FY;
end
```

In keeping with the linearity of the VAR models, we can construct a linear trading model as well. Furthermore, we can choose to make it

TABLE 3.3

Constant Offsets, Autoregressive Coefficients, and Covariance of a VAR(1) Model Applied to Computer Hardware Stocks

| Constant Offsets                              |              | Value      |            | Standard Error |             |
|-----------------------------------------------|--------------|------------|------------|----------------|-------------|
| $\mu_1$                                       |              | 3.88363    |            | 1.15299        |             |
|                                               |              | 0.669367   |            | 0.0970334      |             |
|                                               |              | 1.75474    |            | 0.227636       |             |
|                                               |              | 1.701      |            | 0.249767       |             |
|                                               |              | 1.8752     |            | 0.282581       |             |
| $\phi_{i,j}$                                  | AAPL         | EMC        | HPQ        | NTAP           | SNDK        |
| AAPL                                          | 0.991815     | 0.0735881  | -0.105676  | 0.0359698      | -0.00619303 |
| EMC                                           | -7.15594e-05 | 0.970934   | -0.0103416 | 0.00524778     | 0.00354032  |
| HPQ                                           | -0.00158962  | -0.024093  | 0.965626   | 0.00898799     | 0.00190162  |
| NTAP                                          | -0.000771673 | -0.0409408 | -0.0284176 | 1.00662        | 0.00308001  |
| SNDK                                          | -0.000526824 | -0.0579403 | -0.0309631 | 0.01704        | 0.998657    |
| $\langle \varepsilon_i \varepsilon_j \rangle$ | AAPL         | EMC        | HPQ        | NTAP           | SNDK        |
| AAPL                                          | 36.2559      |            |            |                |             |
| EMC                                           | 1.67571      | 0.256786   |            |                |             |
| HPQ                                           | 3.37592      | 0.449846   | 1.41323    |                |             |
| NTAP                                          | 3.78265      | 0.513747   | 1.20474    | 1.70138        |             |
| SNDK                                          | 4.39542      | 0.522437   | 1.26443    | 1.41357        | 2.17779     |

sector-neutral. We compute the mean predicted return  $\langle r \rangle$  of all the stocks in the industry group every day, and set the target dollar allocation of a stock to be proportional to the difference between its predicted return and the industry group mean,

$$w_i = (r_i - \langle r \rangle) / \sum_j |r_j - \langle r \rangle|. \quad (3.4)$$

We have made sure that the initial gross market value of the portfolio is always \$1. You may notice that this formula looks similar to equation 4.1 in Chan (2013), but it is different. In the formula in my previous book, the returns used are the previous day's returns, and more importantly, we set the proportionality constant to  $-1$  since we assumed mean reversion. The MATLAB code fragment<sup>5</sup> for computing the position (equivalently, dollar allocation) of each stock is

```
retF=(yF-mid)./mid;
sectorRetF=mean(retF, 2);
pos=zeros(size(retF));
pos=(retF-repmat(sectorRetF, [1 size(retF, 2)]))./repmat
    (smartsum(abs(retF-repmat(sectorRetF, [1 size(retF, 2)])), 2),
    [1, size(retF, 2)]);
```

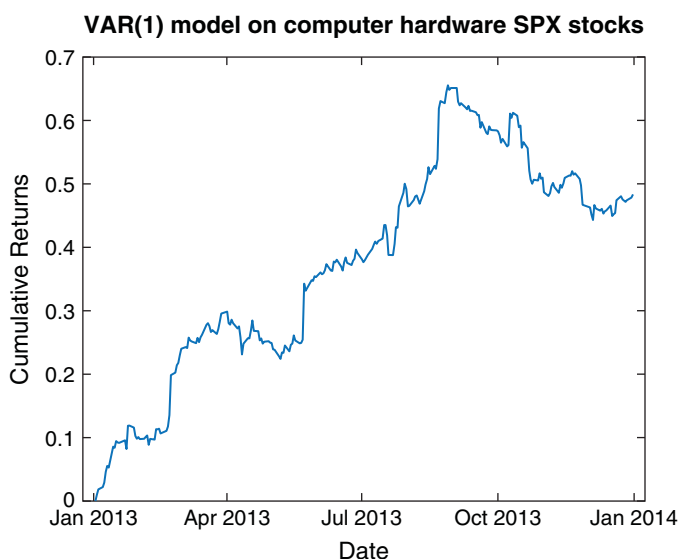
This trading model yields an annualized return of 48 percent, with a Sharpe ratio of 0.9. See Figure 3.3 for its equity curve.

We often want to predict changes in price  $\Delta Y$  instead of price  $Y$  itself. So it is a bit awkward to use the VAR models, and the resulting AR coefficients do not make too much intuitive sense. Fortunately, VAR( $p$ ) can be transformed to a model with  $\Delta Y$  as the dependent variable, and various lagged  $\Delta Y$ 's and  $Y$ 's as the independent variables. This is called the VEC( $q$ ) (vector error correction) model, and is written as

$$\Delta Y(t) = M + CY(t-1) + A_1 \Delta Y(t-1) + \cdots + A_k \Delta Y(t-k) + \varepsilon(t). \quad (3.5)$$

The  $m \times m$  matrix  $C$  in equation 3.5 is called the error correction matrix. To transform the coefficients of VAR( $p$ ) to VEC( $q$ ), first note that  $q = p - 1$ , and we can use the function *vartovec*. Applying this to the VAR model built above for computer hardware stocks:

```
[model_vec, C]=vartovec(model);
```



**FIGURE 3.3** VAR(1) trading strategy applied to computer hardware stocks

we get Table 3.4, which displays the values of  $C$ :

**TABLE 3.4** Error Correction Matrix of a VEC(0) Model Applied to Computer Hardware Stocks

| $C_{i,j}$ | AAPL    | EMC     | HPQ     | NTAP   | SNDK    |
|-----------|---------|---------|---------|--------|---------|
| AAPL      | −0.0082 | 0.0736  | −0.1057 | 0.0360 | −0.0062 |
| EMC       | −0.0001 | −0.0291 | −0.0103 | 0.0052 | 0.0035  |
| HPQ       | −0.0016 | −0.0241 | −0.0344 | 0.0090 | 0.0019  |
| NTAP      | −0.0008 | −0.0409 | −0.0284 | 0.0066 | 0.0031  |
| SNDK      | −0.0005 | −0.0579 | −0.0310 | 0.0170 | −0.0013 |

The values of  $C$  give us a more intuitive understanding of the relationships between the movements of the different stocks. You may notice that except for NTAP, all diagonal elements have negative values. This means that all but NTAP are serially mean reverting, albeit some very weakly.

Equation 3.5 is the same as equation 2.7 in Chan (2013), where it was discussed in connection with the Johansen test for cointegration. Indeed, if the portfolio of computer hardware stocks were cointegrating,  $C$  would give rise to a significantly negative eigenvalue in the Johansen test. But we do not need a cointegrating portfolio to use VEC( $q$ ) for prediction. Some of the stocks could be trending while others are mean reverting, as we saw in Table 3.4.

By the way, if you want to try VAR models on the entire SPX universe instead of just the computer hardware stocks, make sure your computer has an unusually large memory! Also, as mentioned before, these models may behave better if we use log prices instead of prices. (In any case, a log price representation will allow a better connection to the continuous version of VAR and VEC. See Cartea, Jaimungal, and Penalva, 2015, p. 285.)

## ■ State Space Models

The AR, ARMA, VAR, and VEC models we have considered so far all use observable variables (prices of various lags) to predict their future values. However, econometricians have also concocted a class of models with hidden variables, called *states*, which can determine the values of observed variables (though subject to observation noise). These models are called *state space models* (SSM), a linear example of which is the Kalman filter, discussed in Chapter 3 of Chan (2013) and used in Chapter 5 in this book. Though there can be nonlinear state space models, we will discuss only the linear version in this section.

A state space model starts with a linear relationship that specifies the time-evolution of the hidden state variable, usually denoted by  $x$ :

$$x(t) = A(t) * x(t - 1) + B(t) * u(t) \quad (3.6)$$

where  $x$  is an  $m$ -dimensional vector,  $A(t)$  and  $B(t)$  are possibly time-dependent but observable matrices ( $A$  is  $m \times m$ , while  $B$  is  $m \times k$ ), and  $u(t)$  is  $k$ -dimensional Gaussian white noise with zero mean, unit variances, and zero serial and cross correlations. Equation 3.6 is often called the *state transition* equation. The observable variables (also called *measurements*) are related to the hidden variables by another linear equation

$$y(t) = C(t) * x(t) + D(t) * \epsilon(t) \quad (3.7)$$

where  $y$  is an  $n$ -vector,  $C(t)$  and  $D(t)$  are possibly time-dependent but observable matrices ( $C$  is  $n \times m$ , while  $D$  is  $n \times h$ ), and  $\epsilon(t)$  is  $h$ -dimensional Gaussian white noise, also with zero mean, unit variances, and zero serial and cross correlations. Equation 3.7 is often called the *measurement equation*.

What are these hidden variables, and why do we want to hypothesize their existence? An example of a hidden variable is the familiar moving average. Though we usually compute a moving average of prices using a fixed number

of lagged prices and thus making it apparently an observable variable, we can argue that this fixed number of lags is an artificial construction. Also, why not use exponential moving average instead of moving average? The fact that no one can agree on a standard, unique *moving average* variable suggests that it may be treated as a hidden variable. We can give some structure to this hidden variable  $x$  by requiring that it evolves in a particularly simple way:

$$x(t) = x(t - 1) + B * u(t) \quad (3.8)$$

We have assumed  $A(t)$  is the identity matrix, which is of course invariant in time, and  $B$  is an unknown but also time-invariant matrix that determines the covariance of the estimation errors for the moving average  $x$ . (Remember that  $u$  itself has a covariance matrix that is the identity matrix.) Though we had said that  $B$  is supposed to be observable, it can be treated as an unknown parameter(s) to be estimated by applying maximum likelihood estimation on training data. (In other words,  $B$  is “observable” only to the extent that its values are not updated at each time step during Kalman filter updates.)

Given the moving average (plural if the time series is multivariate) of a time series, a trader may hypothesize that the prices are *trending*, and thus the best guess for the observed price at time  $t$  is just the estimated moving average at time  $t$  as well:

$$y(t) = x(t) + D * \varepsilon(t) \quad (3.9)$$

where  $D$  is another unknown and time-invariant matrix to be estimated by MLE.

Let’s see this “moving average” model of equations 3.8 and 3.9 in action by applying it to the same computer hardware stocks’ price series we studied in the section on VAR( $p$ ). We will assume that there are as many hidden state variables (five in total) as there are stocks in the computer hardware industry group. This is what a typical moving average model assumes as well—each price series has its own independent moving average. Furthermore, we assume also that the state noise of one moving average is uncorrelated with any other but each may have a different variance. Hence,  $B$  is a  $5 \times 5$  diagonal matrix with unknown parameters. (Unknown parameters are denoted as NaN as an input to the MATLAB *estimate* function.) Similarly, we will assume the measurement noise of one stock’s price is uncorrelated with another, but each may also have a different variance. Hence,  $D$  is also

a  $5 \times 5$  diagonal matrix with unknown parameters. We could have relaxed this zero-correlation constraint for the state and measurement noises, but this will mean many more variables to estimate, vastly increasing the time it takes for optimization and the danger of overfitting.

The code fragment for using the *estimate* function<sup>6</sup> to generate an estimate of the unknown variances of the state and measurement noises (the parameters in *B* and *D*) are as follows:

```
A=eye(size(y, 2)); % State transition matrix
B=diag(NaN(size(y, 2), 1))
C=eye(size(y, 2)); % Time-invariant measurement matrix
D=diag(NaN(size(y, 2), 1))

model=ssm(A, B, C, D);
param0=randn(2*size(B, 1)^2, 1); % 50 unknown parameters per bar.
model=estimate(model, y(trainset, :), param0);
```

which generates the values shown in Table 3.5.

In this case, the signs of the diagonal elements of the *B* and *D* matrices are immaterial, given that the noises  $u(t)$  and  $\epsilon(t)$  are distributed symmetrically around a zero mean with no cross-correlations. One may also consider applying SSM on log prices instead, so that the Gaussian noise assumption is more reasonable.

| TABLE 3.5   Estimated Values for <i>B</i> and <i>D</i> Matrices (Off-Diagonal Elements Are 0) |            |       |       |       |       |
|-----------------------------------------------------------------------------------------------|------------|-------|-------|-------|-------|
| $B_{i,j}$                                                                                     | $u_1$      | $u_2$ | $u_3$ | $u_4$ | $u_5$ |
| $x_1$                                                                                         | −3.74      |       |       |       |       |
| $x_2$                                                                                         |            | 0.34  |       |       |       |
| $x_3$                                                                                         |            |       | −0.73 |       |       |
| $x_4$                                                                                         |            |       |       | −0.67 |       |
| $x_5$                                                                                         |            |       |       |       | −1.00 |
| $D_{i,j}$                                                                                     | $x_1$      | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
| AAPL                                                                                          | −0.0000454 |       |       |       |       |
| EMC                                                                                           |            | −0.08 |       |       |       |
| HPQ                                                                                           |            |       | 0.22  |       |       |
| NTAP                                                                                          |            |       |       | 0.19  |       |
| SNDK                                                                                          |            |       |       |       | −0.15 |

Once the state transition and measurement equations are fixed, we can use the *filter* function to generate predictions of both the state and observation values.

```
[x, logL, output]=filter(model, y);
```

The  $x(t)$  variable in the output of the *filter* function is the filtered price (moving average) at time  $t$  given observed prices up to time  $t$ . This model generates filtered prices that resemble the observed prices very closely, usually with less than 0.1 percent difference. Given equations (3.8) and (3.9), this also means that our prediction for next day's prices will also closely resemble today's prices. These predicted prices at  $t$  given observed prices can be extracted from *output(t).ForecastedObs*:

```
for t=1:length(output)
    yF(t, :)=output(t).ForecastedObs';
end
```

where we assign the predicted price for time  $t$  to  $yF(t - 1)$ , using the same convention as we did previously. From these predicted prices, we can calculate the predicted returns

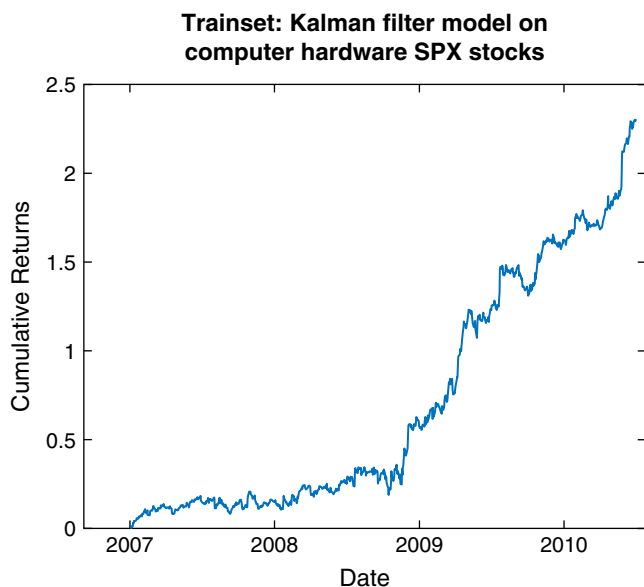
```
retF=(yF-y)./y;
```

Note that *retF(t)* is the predicted return from  $t - 1$  to  $t$ , given the observed price  $y$  at time  $t - 1$ . These predicted returns can be used in the same way as we did in the VAR model to create a sector-neutral trading strategy. We display in Figure 3.4 the cumulative returns of the model on the trainset, and Figure 3.5 displays the cumulative returns on the test set. The degree of overfitting is surprising, given that we merely use the training data to estimate the variances of the state and measurement noises.

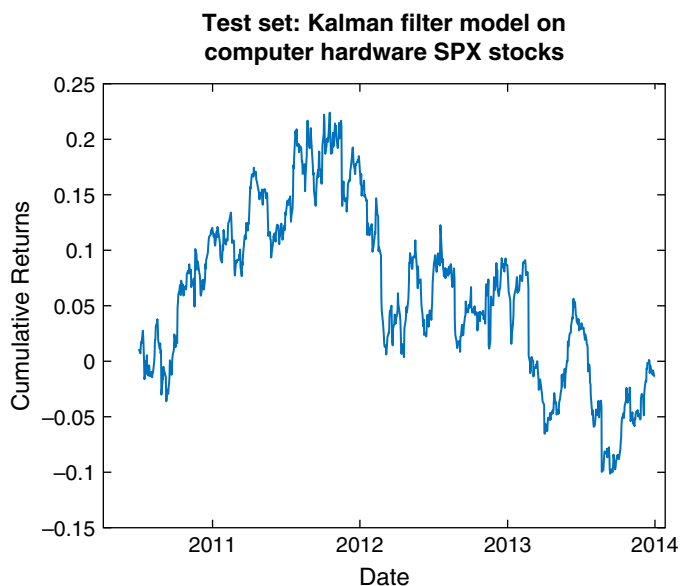
Finding the moving average is not the only way the Kalman filter can be used to predict prices. If we assume trending behavior, we can also use it to find the *slope* of the recent trend in prices, leading to a prediction of the next price assuming the slope persists. This is left as an exercise for the reader.

Using the Kalman filter to make predictions on observations is not the only way to apply it to trading. Estimates of the hidden state itself may be useful—after all, it is supposed to be a moving average. Finding estimates of a hidden variable in the presence of noise is the original meaning of *filtering* and is a well-known concept in signal processing. Besides the Kalman filter, other well-known filters in finance and economics include the Hodrick-Prescott filter and the wavelet filter.





**FIGURE 3.4** Kalman filter trading strategy applied to computer hardware stocks (in-sample)



**FIGURE 3.5** Kalman filter trading strategy applied to computer hardware stocks (out-of-sample)

Another application of Kalman filtering has been discussed in Chan (2013), where it was used to find the best estimates of the hedge ratio between two cointegrated price series. The example given there is the price series of the ETFs EWA (a  $T \times 1$  vector) and EWC (also a  $T \times 1$  vector), which are supposed to be related as

$$[EWC] = [EWA, 1] * \begin{bmatrix} \text{hedge ratio} \\ \text{offset} \end{bmatrix} + \text{noise}$$

But instead of treating the two price series as measurements, we treat EWC as the measurements  $y$ , and EWA augmented with 1s as the time-varying matrix  $C(t)$  in equation 3.7. (The 1s are necessary to allow for the constant offset in the linear regression relationship between EWA and EWC.) We treat the hedge ratio and the constant offset between them as the hidden state  $x$ . Hence, we have

$$x(t) = x(t-1) + B * u(t) \quad (3.10)$$

$$y(t) = C(t) * x(t) + D * \varepsilon(t) \quad (3.11)$$

where  $x$  is a  $2 \times 1$  time-varying vector  $[\text{hedge ratio}, \text{offset}]'$ ,  $y$  is a scalar  $[EWC(t)]$ , and  $C(t)$  is a time-varying  $1 \times 2$  matrix  $[EWA(t), 1]$ . The MATLAB code fragments for these specifications are

```
load('inputData ETF', 'tday', 'syms', 'cl');
idxA=find(strcmp('EWA', syms));
idxC=find(strcmp('EWC', syms));

y=cl(:, idxC);
C=[cl(:, idxA) ones(size(cl, 1), 1)];
A=eye(2);
B=NaN(2);
C=mat2cell(C, ones(size(cl, 1), 1));
D=NaN;
```

where the NaNs indicate unknown parameters. As before, these unknown parameters are estimated by applying the *estimate* function<sup>7</sup> on the trainset from April 26, 2006, to April 9, 2012:

```
trainset=1:1250;
model=ssm(A, B, C(trainset, :), D);
```

and the  $B$  matrix is displayed in Table 3.6, and the scalar  $D$  is estimated as  $-0.08$ . Unlike Table 3.5, we do not impose the constraint that the state noise has zero cross-correlations.

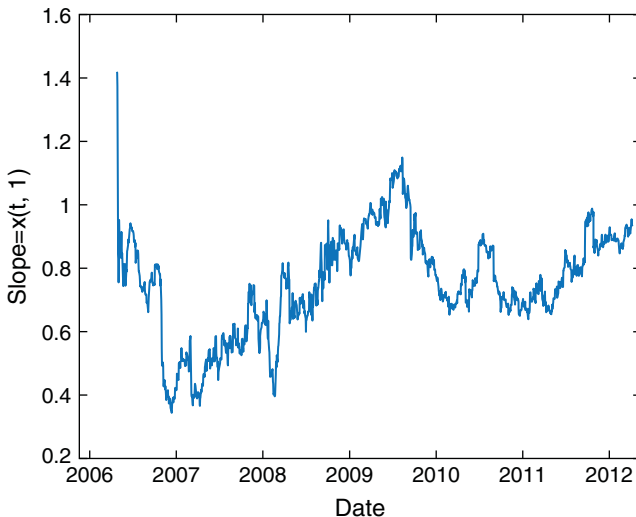
**TABLE 3.6** Estimated Values for  $B$

| $B_{i,j}$ | $u_1$ | $u_2$ |
|-----------|-------|-------|
| $x_1$     | -0.01 | 0.02  |
| $x_2$     | 0.41  | -0.32 |

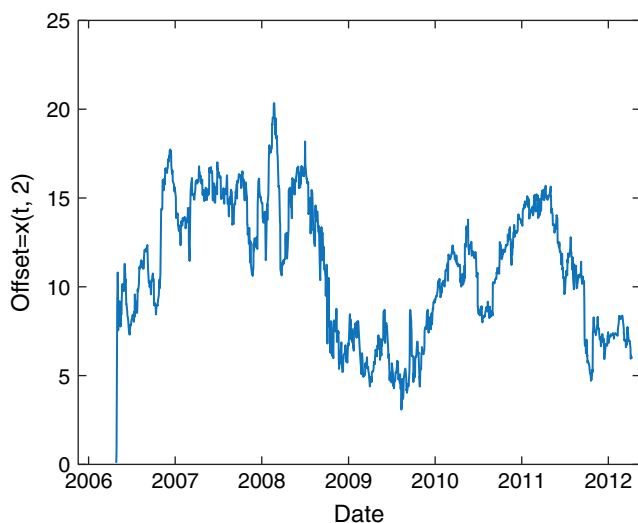
Note that these noise terms are markedly different than the ones we assumed in Box 3.1 of Chan (2013). There, we assumed that the state innovation noises  $\omega_1(t)$  for the hedge ratio and  $\omega_2(t)$  for the offset are uncorrelated, and each has a variance equal to about 0.0001. But here, we have estimated that  $\omega_1(t) = -0.01 * u_1(t) + 0.02 * u_2(t)$  and  $\omega_2(t) = 0.41 * u_1(t) - 0.32 * u_2(t)$ , and given that  $u_1(t)$  and  $u_2(t)$  are assumed to be uncorrelated, the  $\omega$ 's have a covariance matrix

$$\begin{bmatrix} 0.00055 & -0.011 \\ -0.011 & 0.27 \end{bmatrix}.$$

Similarly, instead of arbitrarily setting the variance of the measurement noise  $\epsilon(t)$  to 0.001, we have now estimated that it is  $D^2 = 0.0059$ . Using these estimates and applying the function filter to the data generates estimates of the slope (Figure 3.6) and offset (Figure 3.7) that initially look quite different from Figures 3.5 and 3.6 in Chan (2013), but eventually settle into similar values. We can now apply the same trading strategy that we



**FIGURE 3.6** Kalman filter estimate of the slope between EWC and EWA



**FIGURE 3.7** Kalman filter estimate of the offset between EWC and EWA

described in my previous treatment: buy EWC( $y$ ) if we find that the observed value of  $y$  is smaller than the forecasted value by more than the forecasted standard deviation of the observations, while simultaneously shorting EWA, and vice versa.

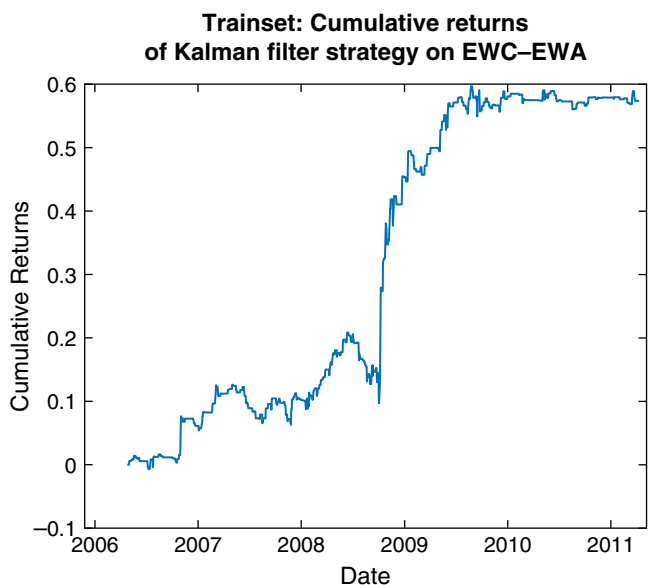
```

yF=NaN(size(y));
ymse=NaN(size(y));
for t=1:length(output)
    yF(t, :)=output(t).ForecastedObs';
    ymse(t, :)=output(t).ForecastedObsCov';
end
e=y-yF; % forecast error
longsEntry=e < -sqrt(ymse); % a long position means
    we should buy EWC
longsExit=e > -sqrt(ymse);

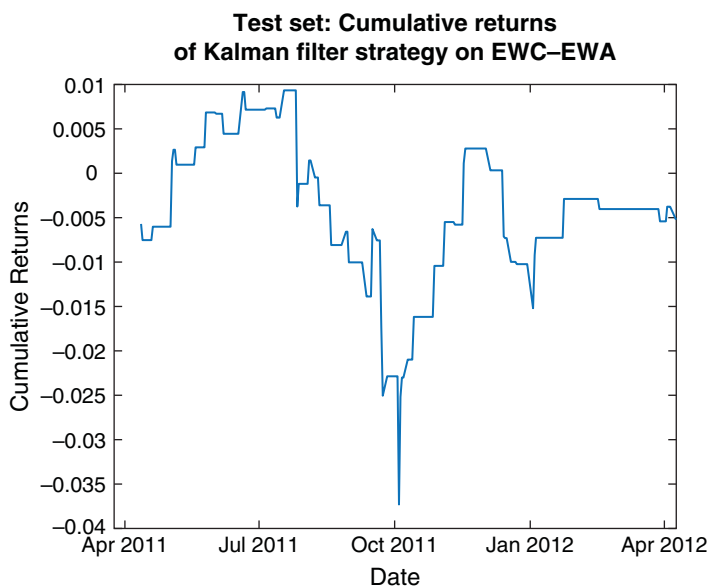
shortsEntry=e > sqrt(ymse);
shortsExit=e < sqrt(ymse);

```

The determination of the actual positions of EWC and EWA are the same as in Chan (2013), and the MATLAB codes can be downloaded as `SSM_beta_EWA_EWC.m`. The cumulative returns of this strategy on the trainset and the test set are depicted in Figures 3.8 and 3.9, respectively.



**FIGURE 3.8** Kalman filter trading strategy applied to EWC-EWA (in-sample)



**FIGURE 3.9** Kalman filter trading strategy applied to EWC-EWA (out-of-sample)

We can see that the equity curve has started to flatten even during the latter part of the trainset. This could have been a result of regime change, where EWA and EWC have fallen out of cointegration, or more likely, a result of overfitting the noise covariance matrix  $B$ .

## ■ Summary

Time-series analysis is the first technique one should try when confronted with a brand-new financial instrument or market, and we have not yet developed any intuition about it. We have surveyed some of the most popular linear models of time series that have found their way into many quantitative traders' strategies. Despite their linearity, there are often many parameters that need to be estimated, and so overfitting is a constant danger. This is especially true for state space models, where there is an extra hidden variable with its own dynamics that need to be estimated. A successful application of these methods to strategy building will involve imposing judicious constraints to reduce the number of unknown parameters. A popular constraint in the case of the ARMA or VAR models would be to limit the number of lags to 1, and in the case of the SSM, the assumption of zero cross correlations for the noises. Beyond imposing constraints, training the models on a large amount of data is the ultimate cure, pointing to their promise in intraday trading.

## ■ Exercises

- 3.1. Show that if  $Y(t)$  in the AR(1) process in equation 3.1 is weakly stationary, then  $|\phi| < 1$ . *Hint:* Consider the variance of  $Y(t)$ .
- 3.2. In the section on AR( $p$ ), we described a backtest on AUD.USD using an AR(1) that achieved a CAGR of 158 percent using midprices. The same .mat data set also contains bid and ask quotes separately. Backtest the same strategy assuming we use market orders only. What is the resulting CAGR?
- 3.3. Using MATLAB's *arima* and *estimate* functions, verify that using ARIMA( $p$ , 0,  $q$ ) to model log returns of AUD.USD gives the same autoregressive coefficients as using ARIMA( $p$ , 1,  $q$ ) to model log prices. Show also that the best estimates for  $p$  and  $q$  are 1 and 9, respectively.

- 3.4. Apply the VAR model to EWA and EWC, and generate daily buy/sell trading signals when the predicted daily return is positive/negative. Assuming we always trade \$1 per ETF, what is the CAGR and Sharpe ratio? Are there times when the trading signals for both ETFs have the same sign?
- 3.5. Comparing the *moving average* generated by equations (3.8) and (3.9) with an  $N$ -day exponential moving average (e.g., see [en.wikipedia.org/wiki/Moving\\_average](http://en.wikipedia.org/wiki/Moving_average)), what is the  $N$  that best fits our estimated state variable? What constraint(s) would you need to apply to the  $B$  or  $D$  matrices in equations 3.8 and 3.9 in order to enforce a larger  $N$ ?
- 3.6. If you assume that  $B$  is diagonal in equation 3.9, are you able to backtest the Kalman filter trading strategy for EWC vs. EWA with a CAGR of 26.2 percent and a Sharpe ratio of 2.4 using data from April 26, 2006, to April 9, 2012? (These are the results we obtained in Chan, 2013.)
- 3.7. Apply VAR and VEC on computer hardware stocks as shown in the section on VAR( $p$ ) using log prices instead of prices. Do the out-of-sample returns and Sharpe ratio improve?
- 3.8. Instead of using the Kalman filter to find the *moving average* of prices, use it to find the slope of the recent price trend. Assuming that this slope persists into the future, backtest a trending strategy on, for example, the computer hardware stocks.

## ■ Endnotes

1. A time series is strictly stationary if all aspects of its behavior are unchanged by shifts in time (Ruppert and Matteson, 2015). A weakly stationary time series only requires that the mean and variance are unchanged. If this is a multivariate time series, the covariance also needs to be unchanged.
2. This complete program can be downloaded as `buildAR1.m`.
3. The complete code can be downloaded as `buildARp_AUDUSD.m`.
4. The complete code can be downloaded as `buildVAR_findP_stocks.m`.
5. The complete code can be downloaded as `buildVAR1_sectorNeutral_computerHardware.m`.
6. The complete code can be downloaded as `SSM_MA_computerHardware_diag.m`.
7. The complete code can be downloaded as `SSM_beta_EWA_EWC.m`.

